EXPLORATORY DATA ANALYSIS
&
ADVANCED ANALYTICS
PROJECT

STEPS TO FOLLOW

EXPLORATORY DATA ANALYSIS

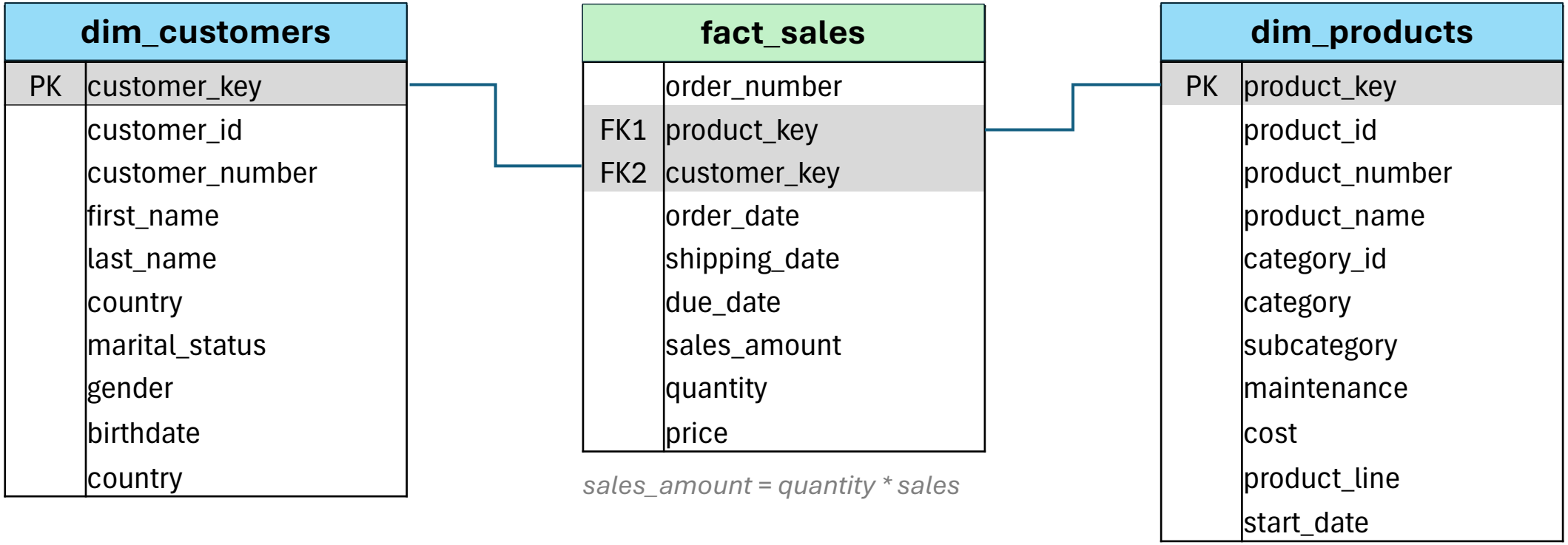
- Database Exploration
- Dimentions Exploration
- Date Exploration
- Measures Exploration
- Magnitude Analysis
- Ranking

STEPS TO FOLLOW

ADVANCED ANALYTICS

- Trends Analysis (Change over time)
- Cumulative Analysis
- Performance Analysis
- Part to Whole (Proportional)
- Segmentation
- Reporting

DATABASE EXPLORATION



DIMENTIONS EXPLORATION

Explore and identify different dimensions valid in the database like Country that customers live in, category and sub-categories of the products and many more...

```sql
-- Retrieve a list of unique countries
--from which customers originate
SELECT DISTINCT
    country
FROM gold.dim_customers
ORDER BY country;
```

	country
1	AUSTRALIA
2	CANADA
3	FRANCE
4	GERMANY
5	n/a
6	UK
7	USA

```sql
-- Retrieve a list of unique categories,
--subcategories, and products
SELECT DISTINCT
    category,
    subcategory,
    product_name
FROM gold.dim_products
ORDER BY category, subcategory, product_name;
```

	category	subcategory	product_name
25	Accessories	Pumps	Mountain Pump
26	Accessories	Tires and Tubes	HL Mountain Tire
27	Accessories	Tires and Tubes	HL Road Tire
28	Accessories	Tires and Tubes	LL Mountain Tire
29	Accessories	Tires and Tubes	LL Road Tire
30	Accessories	Tires and Tubes	ML Mountain Tire
31	Accessories	Tires and Tubes	ML Road Tire
32	Accessories	Tires and Tubes	Mountain Tire Tube
33	Accessories	Tires and Tubes	Patch Kit/8 Patches
34	Accessories	Tires and Tubes	Road Tire Tube
35	Accessories	Tires and Tubes	Touring Tire
36	Accessories	Tires and Tubes	Touring Tire Tube
37	Bikes	Mountain Bikes	Mountain-100 Black- 38
38	Bikes	Mountain Bikes	Mountain-100 Black- 42

DATE EXPLORATION

Identify the boundries and timespan of the data by exploring the earliest and latest dates

```sql
-- Determine the first and last order date and the total duration in months
SELECT
    MIN(order_date) AS first_order_date,
    MAX(order_date) AS last_order_date,
    DATEDIFF(MONTH, MIN(order_date), MAX(order_date)) AS order_range_months
FROM gold.fact_sales;

-- Find the youngest and oldest customer based on birthdate
SELECT
    MIN(birthdate) AS oldest_birthdate,
    DATEDIFF(YEAR, MIN(birthdate), GETDATE()) AS oldest_age,
    MAX(birthdate) AS youngest_birthdate,
    DATEDIFF(YEAR, MAX(birthdate), GETDATE()) AS youngest_age
FROM gold.dim_customers;
```

MEASURES EXPLORATION

Calculate the general key metrics of the business like Total Sales, Average Product Price, Total Quantity Sold

```sql
-- Generate a Report that shows all key metrics of the business
SELECT 'Total Sales' AS measure_name, SUM(sales_amount) AS measure_value FROM gold.fact_sales
UNION ALL
SELECT 'Total Quantity', SUM(quantity) FROM gold.fact_sales
UNION ALL
SELECT 'Average Price', AVG(price) FROM gold.fact_sales
UNION ALL
SELECT 'Total Orders', COUNT(DISTINCT order_number) FROM gold.fact_sales
UNION ALL
SELECT 'Total Products', COUNT(DISTINCT product_name) FROM gold.dim_products
UNION ALL
SELECT 'Total Customers', COUNT(customer_key) FROM gold.dim_customers;
```

	measure_name	measure_value
1	Total Sales	29356250
2	Total Quantity	60423
3	Average Price	486
4	Total Orders	27659
5	Total Products	295
6	Total Customers	18484

MAGNITUDE ANALYSIS

Compare the measure values by categories to understand the importance of different categories like: Total Sales by Country, Average Product Price by Category, etc...

```sql
-- Find total customers by countries
SELECT
    country,
    COUNT(customer_key) AS total_customers
FROM gold.dim_customers
GROUP BY country
ORDER BY total_customers DESC;
```

	country	total_customers
1	USA	7482
2	AUSTRALIA	3591
3	UK	1913
4	FRANCE	1810
5	GERMANY	1780
6	CANADA	1571
7	n/a	337

```sql
-- What is the total revenue generated
--for each category?
SELECT
    p.category,
    SUM(f.sales_amount) AS total_revenue
FROM gold.fact_sales f
LEFT JOIN gold.dim_products p
    ON p.product_key = f.product_key
GROUP BY p.category
ORDER BY total_revenue DESC;
```

	category	total_revenue
1	Bikes	28316272
2	Accessories	700262
3	Clothing	339716

```sql
-- What is the total revenue
--generated by each customer?
SELECT
    c.customer_key,
    c.first_name,
    c.last_name,
    SUM(f.sales_amount) AS total_revenue
FROM gold.fact_sales f
LEFT JOIN gold.dim_customers c
    ON c.customer_key = f.customer_key
GROUP BY
    c.customer_key,
    c.first_name,
    c.last_name
ORDER BY total_revenue DESC;
```

	customer_key	first_name	last_name	total_revenue
1	1302	Nichole	Nara	13294
2	1133	Kaitlyn	Henderson	13294
3	1309	Margaret	He	13268
4	1132	Randall	Dominguez	13265
5	1301	Adriana	Gonzalez	13242
6	1322	Rosa	Hu	13215

RANKING ANALYSIS

Finding the order of values of dimensions by measure to distinguish top and buttom performers

```sql
-- Which 5 products Generating the Highest Revenue?
SELECT *
FROM (
    SELECT
        p.product_name,
        SUM(f.sales_amount) AS total_revenue,
        RANK() OVER (ORDER BY SUM(f.sales_amount) DESC) AS rank_products
    FROM gold.fact_sales f
    LEFT JOIN gold.dim_products p
        ON p.product_key = f.product_key
    GROUP BY p.product_name
) AS ranked_products
WHERE rank_products <= 5;
```

	product_name	total_revenue	rank_products
1	Mountain-200 Black- 46	1373454	1
2	Mountain-200 Black- 42	1363128	2
3	Mountain-200 Silver- 38	1339394	3
4	Mountain-200 Silver- 46	1301029	4
5	Mountain-200 Black- 38	1294854	5

TREND ANALYSIS

Analyse how measures evolve over time to track trends and seasonality of the metrics like Total Sales by Month, Average Cost by year, etc

```sql
-- Analyse sales performance over time
-- Quick Date Functions
SELECT
    YEAR(order_date) AS order_year,
    MONTH(order_date) AS order_month,
    SUM(sales_amount) AS total_sales,
    COUNT(DISTINCT customer_key) AS total_customers,
    SUM(quantity) AS total_quantity
FROM gold.fact_sales
WHERE order_date IS NOT NULL
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY YEAR(order_date), MONTH(order_date);
```

	order_year	order_month	total_sales	total_customers	total_quantity
1	2010	12	43419	14	14
2	2011	1	469795	144	144
3	2011	2	466307	144	144
4	2011	3	485165	150	150
5	2011	4	502042	157	157
6	2011	5	561647	174	174
7	2011	6	737793	230	230
8	2011	7	596710	188	188
9	2011	8	614516	193	193
10	2011	9	603047	185	185
11	2011	10	708164	221	221
12	2011	11	660507	208	208
13	2011	12	669395	222	222
14	2012	1	495363	252	252

CUMULATIVE ANALYSIS

Aggregating data cumulatively. It helps to understand if metrics grow or decline over time like Running Total Sales by Month, Moving Average of Customer Satisfaction by Month

```sql
-- Calculate the total sales per month
-- and the running total of sales over time
SELECT
    order_date,
    total_sales,
    SUM(total_sales) OVER (ORDER BY order_date) AS running_total_sales,
    AVG(avg_price) OVER (ORDER BY order_date) AS moving_average_price
FROM
(
    SELECT
        DATETRUNC(year, order_date) AS order_date,
        SUM(sales_amount) AS total_sales,
        AVG(price) AS avg_price
    FROM gold.fact_sales
    WHERE order_date IS NOT NULL
    GROUP BY DATETRUNC(year, order_date)
) t
```

	order_date	total_sales	running_total_sales	moving_average_price
1	2010-01-01	43419	43419	3101
2	2011-01-01	7075088	7118507	3146
3	2012-01-01	5842231	12960738	2670
4	2013-01-01	16344878	29305616	2080
5	2014-01-01	45642	29351258	1668

PERFORMANCE ANALYSIS

Compare the current value with a target value to measure success and evaluate the performance.
Compare current month sales by previous year's same month sales.

```sql
/* Analyze the yearly performance of products by comparing their sales
to both the average sales performance of the product and the previous year's sales */
WITH yearly_product_sales AS (
    SELECT
        YEAR(f.order_date) AS order_year,
        p.product_name,
        SUM(f.sales_amount) AS current_sales
    FROM gold.fact_sales f
    LEFT JOIN gold.dim_products p
        ON f.product_key = p.product_key
    WHERE f.order_date IS NOT NULL
    GROUP BY
        YEAR(f.order_date),
        p.product_name
)
SELECT
    order_year,
    product_name,
    current_sales,
    AVG(current_sales) OVER (PARTITION BY product_name) AS avg_sales,
    current_sales - AVG(current_sales) OVER (PARTITION BY product_name) AS diff_avg,
    CASE
        WHEN current_sales - AVG(current_sales) OVER (PARTITION BY product_name) > 0 THEN 'Above Avg'
        WHEN current_sales - AVG(current_sales) OVER (PARTITION BY product_name) < 0 THEN 'Below Avg'
        ELSE 'Avg'
    END AS avg_change,
    -- Year-over-Year Analysis
    LAG(current_sales) OVER (PARTITION BY product_name ORDER BY order_year) AS py_sales,
    current_sales - LAG(current_sales) OVER (PARTITION BY product_name ORDER BY order_year) AS diff_py,
    CASE
        WHEN current_sales - LAG(current_sales) OVER (PARTITION BY product_name ORDER BY order_year) > 0 THEN 'Increase'
        WHEN current_sales - LAG(current_sales) OVER (PARTITION BY product_name ORDER BY order_year) < 0 THEN 'Decrease'
        ELSE 'No Change'
    END AS py_change
FROM yearly_product_sales
ORDER BY product_name, order_year;
```

	order_year	product_name	current_sales	avg_sales	diff_avg	avg_change	py_sales	diff_py	py_change
1	2012	All-Purpose Bike Stand	159	13197	-13038	Below Avg	NULL	NULL	No Change
2	2013	All-Purpose Bike Stand	37683	13197	24486	Above Avg	159	37524	Increase
3	2014	All-Purpose Bike Stand	1749	13197	-11448	Below Avg	37683	-35934	Decrease
4	2012	AWC Logo Cap	72	6570	-6498	Below Avg	NULL	NULL	No Change
5	2013	AWC Logo Cap	18891	6570	12321	Above Avg	72	18819	Increase
6	2014	AWC Logo Cap	747	6570	-5823	Below Avg	18891	-18144	Decrease
7	2013	Bike Wash - Dissolver	6960	3636	3324	Above Avg	NULL	NULL	No Change
8	2014	Bike Wash - Dissolver	312	3636	-3324	Below Avg	6960	-6648	Decrease
9	2013	Classic Vest- L	11968	6240	5728	Above Avg	NULL	NULL	No Change
10	2014	Classic Vest- L	512	6240	-5728	Below Avg	11968	-11456	Decrease
11	2013	Classic Vest- M	11840	6368	5472	Above Avg	NULL	NULL	No Change
12	2014	Classic Vest- M	896	6368	-5472	Below Avg	11840	-10944	Decrease

PART TO WHOLE ANALYSIS

Analyze an individual part's performance compared to overall to understand the impact of individual parts like percentage of a category in total sales

```sql
-- Which categories contribute the most to overall sales?
WITH category_sales AS (
    SELECT
        p.category,
        SUM(f.sales_amount) AS total_sales
    FROM gold.fact_sales f
    LEFT JOIN gold.dim_products p
        ON p.product_key = f.product_key
    GROUP BY p.category
)
SELECT
    category,
    total_sales,
    SUM(total_sales) OVER () AS overall_sales,
    ROUND((CAST(total_sales AS FLOAT) / SUM(total_sales) OVER ()) * 100, 2) AS percentage_of_total
FROM category_sales
ORDER BY total_sales DESC;
```

	category	total_sales	overall_sales	percentage_of_total
1	Bikes	28316272	29356250	96,46
2	Accessories	700262	29356250	2,39
3	Clothing	339716	29356250	1,16

SEGMENTATION

Regroup the data based on a specific range to understand the correlation between measures like Total Number of Customers by Age Groups, Customers by spending

```sql
/*Group customers into three segments based on their spending behavior:
    - VIP: Customers with at least 12 months of history and spending more than €5,000.
    - Regular: Customers with at least 12 months of history but spending €5,000 or less.
    - New: Customers with a lifespan less than 12 months.
And find the total number of customers by each group
*/
WITH customer_spending AS (
    SELECT
        c.customer_key,
        SUM(f.sales_amount) AS total_spending,
        MIN(order_date) AS first_order,
        MAX(order_date) AS last_order,
        DATEDIFF(month, MIN(order_date), MAX(order_date)) AS lifespan
    FROM gold.fact_sales f
    LEFT JOIN gold.dim_customers c
        ON f.customer_key = c.customer_key
    GROUP BY c.customer_key
)
SELECT
    customer_segment,
    COUNT(customer_key) AS total_customers
FROM (
    SELECT
        customer_key,
        CASE
            WHEN lifespan >= 12 AND total_spending > 5000 THEN 'VIP'
            WHEN lifespan >= 12 AND total_spending <= 5000 THEN 'Regular'
            ELSE 'New'
        END AS customer_segment
    FROM customer_spending
) AS segmented_customers
GROUP BY customer_segment
ORDER BY total_customers DESC;
```

	customer_segment	total_customers
1	New	14631
2	Regular	2198
3	VIP	1655

BUILDING REPORTS1 – customer report

Build a customer report that brings together many metrics about customers (as a view in the database)

1. Essential fields like names, ages, and transaction details.

2. Customer Segmentation based on spending and age group (VIP, Regular, New)

3. Customer-level metrics:
   - Total Orders
   - Total Sales
   - Total Items Purchased
   - Total Products Purchased
   - Lifespan (in months, how long is the customer in our database?)
4. Calculate Business KPIs:
   -Number of months since last order
   - average number of orders
   - average monthly spending

CUSTOMER REPORT

	customer_key	customer_number	customer_name	age	age_group	customer_segment	last_order_date	recency	total_orders	total_sales	total_quantity	lifespan	avg_order_value	avg_monthly_spend
1	1	AW00011000	Jon Yang	55	50 and above	VIP	2013-05-03	155	3	8249	8	8	2749	294
2	2	AW00011001	Eugene Huang	50	50 and above	VIP	2013-12-10	148	3	6384	11	10	2128	182
3	3	AW00011002	Ruben Torres	55	50 and above	VIP	2013-02-23	158	3	8114	4	4	2704	324
4	4	AW00011003	Christy Zhu	53	50 and above	VIP	2013-05-10	155	3	8139	9	9	2713	280
5	5	AW00011004	Elizabeth Johnson	47	40-49	VIP	2013-05-01	155	3	8196	6	6	2732	292
6	6	AW00011005	Julio Ruiz	50	50 and above	VIP	2013-05-02	155	3	8121	6	6	2707	280
7	7	AW00011006	Janet Alvarez	50	50 and above	VIP	2013-05-14	155	3	8119	5	5	2706	289
8	8	AW00011007	Marco Mehta	57	50 and above	VIP	2013-03-19	157	3	8211	8	8	2737	315
9	9	AW00011008	Rob Verhoff	51	50 and above	VIP	2013-03-02	157	3	8106	7	7	2702	311
10	10	AW00011009	Shannon Carlson	57	50 and above	VIP	2013-05-09	155	3	8091	5	5	2697	288
11	11	AW00011010	Jacquelyn Suarez	57	50 and above	VIP	2013-05-23	155	3	8088	4	4	2696	288
12	12	AW00011011	Curtis Lu	57	50 and above	VIP	2013-03-19	157	3	8133	4	4	2711	301
13	13	AW00011012	Lauren Walker	47	40-49	New	2013-10-15	150	2	81	5	5	40	11
14	14	AW00011013	Ian Jenkins	47	40-49	New	2014-01-21	147	2	114	5	5	57	12
15	15	AW00011014	Sydney Bennett	53	50 and above	New	2013-04-30	156	2	138	6	5	69	138
16	16	AW00011015	Chloe Young	42	40-49	New	2013-01-18	159	1	2501	3	3	2501	2501

```sql
SELECT
age_group,
COUNT(customer_number) AS total_customers,
SUM(total_sales) AS total_sales
FROM gold.report_customers
GROUP BY age_group
```

	age_group	total_customers	total_sales
1	50 and above	12846	20686539
2	40-49	5636	8664719

BUILDING REPORTS 2 – product report

Build a product report that brings together many metrics about products (as a view in the database)

1. Present product name, category, subcategory, and cost.
2. Product Segmentation based on revenue to identify High-Performers, Mid-Range, or Low-Performers.
3. Aggregates product-level metrics:
   - Total Number of Orders
   - Total Sales
   - Total Quantity Ordered
   - Number of Total Unique Customers
   - Lifespan (in months, since how long the product exists)
4. Calculates KPIs:
   - Number of months since last sale
   - Average Revenue per Order (AOR)
   - Average Monthly Revenue

PRODUCT REPORT

	prod...	product_name	category	subcategory	cost	last_sale_date	recency_...	product_segment	lifespan	total_orders	total_sales	total_quant...	total_custo...	avg_selling_pri...	avg_order_rev...	avg_monthly_rev...
1	3	Mountain-100 ...	Bikes	Mountain ...	1898	2011-12-27	172	High-Performer	11	49	165375	49	49	3375	3375	15034
2	4	Mountain-100 ...	Bikes	Mountain ...	1898	2011-12-27	172	High-Performer	11	45	151875	45	45	3375	3375	13806
3	5	Mountain-100 ...	Bikes	Mountain ...	1898	2011-12-21	172	High-Performer	11	60	202500	60	60	3375	3375	18409
4	6	Mountain-100 ...	Bikes	Mountain ...	1898	2011-12-26	172	High-Performer	12	57	192375	57	57	3375	3375	16031
5	7	Mountain-100 ...	Bikes	Mountain ...	1912	2011-12-22	172	High-Performer	12	58	197200	58	58	3400	3400	16433
6	8	Mountain-100 ...	Bikes	Mountain ...	1912	2011-12-28	172	High-Performer	11	42	142800	42	42	3400	3400	12981
7	9	Mountain-100 ...	Bikes	Mountain ...	1912	2011-12-12	172	High-Performer	12	49	166600	49	49	3400	3400	13883
8	10	Mountain-100 ...	Bikes	Mountain ...	1912	2011-12-23	172	High-Performer	11	36	122400	36	36	3400	3400	11127
9	16	Road-150 Re...	Bikes	Road Bikes	2171	2011-12-28	172	High-Performer	12	281	1005418	281	281	3578	3578	83784
10	17	Road-150 Re...	Bikes	Road Bikes	2171	2011-12-28	172	High-Performer	12	337	1205786	337	337	3578	3578	100482
11	18	Road-150 Re...	Bikes	Road Bikes	2171	2011-12-27	172	High-Performer	12	302	1080556	302	302	3578	3578	90046
12	19	Road-150 Re...	Bikes	Road Bikes	2171	2011-12-27	172	High-Performer	12	295	1055510	295	295	3578	3578	87959
13	20	Road-150 Re...	Bikes	Road Bikes	2171	2011-12-28	172	High-Performer	12	336	1202208	336	336	3578	3578	100184
14	43	Road-650 Bla...	Bikes	Road Bikes	487	2012-12-26	160	Mid-Range	23	63	47565	63	63	755	755	2068
15	44	Road-650 Bla...	Bikes	Road Bikes	487	2012-12-25	160	Mid-Range	21	60	45552	60	60	759,2	759	2169
16	45	Road-650 Bla...	Bikes	Road Bikes	487	2012-12-19	160	High-Performer	23	89	66915	89	89	751,9	751	2909
17	46	Road-650 Bla...	Bikes	Road Bikes	487	2012-12-18	160	High-Performer	23	76	57996	76	76	763,1	763	2521
18	47	Road-650 Bla...	Bikes	Road Bikes	487	2012-12-12	160	High-Performer	22	76	57156	76	76	752,1	752	2598
19	48	Road-650 Bla...	Bikes	Road Bikes	487	2012-12-18	160	Mid-Range	24	65	49047	65	65	754,6	754	2043
20	49	Road-650 Re...	Bikes	Road Bikes	487	2012-12-25	160	High-Performer	23	72	54528	72	72	757,3	757	2370
21	50	Road-650 Re...	Bikes	Road Bikes	487	2012-12-27	160	High-Performer	23	88	66720	88	88	758,2	758	2900
22	51	Road-650 Re...	Bikes	Road Bikes	487	2012-12-21	160	Mid-Range	24	61	46083	61	61	755,5	755	1920